

Minimax Algorithm for Gaming (AI)

Aim

To implement the Minimax algorithm to determine the optimal move for the Maximizer in a two-player, zero-sum game represented as a game tree.

Objectives

- To understand adversarial search in game-playing AI.
- To represent the game as a tree of possible moves and outcomes.
- To alternate between Maximizer and Minimizer at each level.
- To determine the optimal achievable value for the Maximizer.

Algorithm

- 1 Start.
- 2 Represent the game as a tree with leaf nodes holding utility values.
- 3 If the current node is a leaf (terminal state), return its value.
- 4 If it is the Maximizer's turn, return the maximum value among child nodes.
- 5 If it is the Minimizer's turn, return the minimum value among child nodes.
- 6 Recursively evaluate all nodes from the leaves up to the root.
- 7 Determine the optimal value obtainable by the Maximizer at the root.
- 8 Display the optimal value.
- 9 Stop.

Source Code (Python)

```
tree = [3, 5, 2, 9, 12, 5, 23, 23]
def minimax(depth, node_index, is_max, values, max_depth):
    if depth == max_depth:
        return values[node_index]
    if is_max:
        return max(minimax(depth+1, node_index*2, False, values, max_depth),
                    minimax(depth+1, node_index*2+1, False, values, max_depth))
    else:
        return min(minimax(depth+1, node_index*2, True, values, max_depth),
                    minimax(depth+1, node_index*2+1, True, values, max_depth))

import math
max_depth = int(math.log2(len(tree)))
print("Leaf node values (game tree):", tree)
print(f"Tree depth: {max_depth}\n")
optimal_value = minimax(0, 0, True, tree, max_depth)
print(f"The optimal value for the Maximizer is: {optimal_value}")
```

Output

```
Leaf node values (game tree): [3, 5, 2, 9, 12, 5, 23, 23]
Tree depth: 3
The optimal value for the Maximizer is: 12
```

Result

The Minimax algorithm was successfully implemented on the given game tree. The optimal value achievable by the Maximizer was found to be 12.